Marmara University
Engineering Faculty
Electrical and Electronics Engineering
EE 4077: Fundamentals of Machine Learning
Final Exam Makeup (Spring, 2024)

1. Kernels [25 points]

(1) [5 pts] What is the primary motivation for using the kernel trick in machine learning algorithms?

(2) [10pts] Prove that for every design matrix $X \in \mathbb{R}^{n \times d}$, the corresponding kernel matrix is positive semidefinite.

(3) [5 pts] Suppose that a regression algorithm contains the following line of code.

$$\mathbf{w} \leftarrow \mathbf{w} + X^\top M X X^\top \mathbf{u}$$

Here, $X \in \mathbb{R}^{n \times d}$ is the design matrix, $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, $M \in \mathbb{R}^{n \times n}$ is a matrix unrelated to X, and $\mathbf{u} \in \mathbb{R}^n$ is a vector unrelated to X. We want to derive a dual version of the algorithm in which we express the weights $\mathbf{w}$ as a linear combination of samples X_i (rows of X) and a dual weight vector $\mathbf{a}$ contains the coefficients of that linear combination. Rewrite the line of code in its dual form so that it updates $\mathbf{a}$ correctly (and so that $\mathbf{w}$ does not appear).

(4) [5 pts] Can this line of code for updating $\mathbf{a}$ be kernelized? If so, show how. If not, explain why.

2. PCA [25 points]

You are given a design matrix $X = \begin{bmatrix} 6 & -4 \\ -3 & 5 \\ -2 & 6 \\ 7 & -3 \end{bmatrix}$. Let's use PCA to reduce the dimension from 2 to 1.

(1) [8 pts] Compute the covariance matrix for the sample points. (Warning: Observe that X is not centered.) Then compute the **unit** eigenvectors, and the corresponding eigenvalues, of the covariance matrix. *Hint:* If you graph the points, you can probably guess the eigenvectors (then verify that they really are eigenvectors).

(2) [8 pts] Suppose we use PCA to project the sample points onto a one-dimensional space. What one-dimensional subspace are we projecting onto? For each of the four sample points in X (not the centered version of X!), write the coordinate (in principal coordinate space, not in $\mathbb{R}^2$) that the point is projected to.

(3) [9 pts] Given a design matrix X that is taller than it is wide, prove that every right singular vector of X with singular value σ is an eigenvector of the covariance matrix with eigenvalue σ^2.

3. Convolutional Neural Networks [25 points]

(a) [5 pts] Consider a convolutional neural network for reading the handwritten MNIST letters, which are 28×28 images. Suppose the first hidden layer is a convolutional layer with 20 different 5×5 filters, applied to the input image with a stride of 1 (i.e., every filter is applied to *every* 5×5 patch of the image, with patches allowed to overlap). Each filter has a bias weight. How many weights (parameters) does this layer use?

(b) [10 pts] Let X be an $n \times d$ design matrix representing n sample points in $\mathbb{R}^d$. Let $X = UDV^\top$ be the singular value decomposition of X. We stated in lecture that row i of the matrix UD gives the coordinates of sample point X_i in principal coordinates space, i.e., $X_i \cdot v_j$ for each j, where X_i is the ith row of X and v_j is the jth column of V. Show that this is true.

(c) [10 pts] Let $x, y \in \mathbb{R}^d$ be two points (e.g., sample or test points). Consider the function $k(x, y) = x^\top \text{rev}(y)$ where $\text{rev}(y)$ reverses the order of the components in y. For example, $\text{rev}\left(\begin{bmatrix}1\\2\\3\end{bmatrix}\right) = \begin{bmatrix}3\\2\\1\end{bmatrix}$. Show that k *cannot* be a valid kernel function.

Hint: remember how the kernel function is defined, and show a simple two-dimensional counterexample.

4. Neural Networks [25 points]

You want to train a neural network to drive a car. Your training data consists of grayscale 64×64 pixel images. The training labels include the human driver's steering wheel angle in degrees and the human driver's speed in miles per hour. Your neural network consists of an input layer with $64 \times 64 = 4{,}096$ units, a hidden layer with 2,048 units, and an output layer with 2 units (one for steering angle, one for speed). You use the ReLU activation function for the hidden units and no activation function for the outputs (or inputs).

(1) [5 pts] Calculate the number of parameters (weights) in this network. You can leave your answer as an expression. Be sure to account for the bias terms.

(2) [5 pts] You train your network with the cost function $J = \frac{1}{2}|\mathbf{y} - \mathbf{z}|^2$. Use the following notation.

- $\mathbf{x}$ is a training image (input) vector with a 1 component appended to the end, $\mathbf{y}$ is a training label (input) vector, and $\mathbf{z}$ is the output vector. All vectors are column vectors.
- $r(\gamma) = \max\{0, \gamma\}$ is the ReLU activation function, $r'(\gamma)$ is its derivative (1 if $\gamma > 0$, 0 otherwise), and $r(\mathbf{v})$ is $r(\cdot)$ applied component-wise to a vector.
- $\mathbf{g}$ is the vector of hidden unit values before the ReLU activation functions are applied, and $\mathbf{h} = r(\mathbf{g})$ is the vector of hidden unit values after they are applied (but we append a 1 component to the end of $\mathbf{h}$).
- V is the weight matrix mapping the input layer to the hidden layer; $\mathbf{g} = V\mathbf{x}$.
- W is the weight matrix mapping the hidden layer to the output layer; $\mathbf{z} = W\mathbf{h}$.

Derive $\partial J / \partial W_{ij}$.

(3) [5 pt] Write $\partial J / \partial W$ as an outer product of two vectors. $\partial J / \partial W$ is a matrix with the same dimensions as W: it's just like a gradient, except that W and $\partial J / \partial W$ are matrices rather than vectors.

(4) [10 pts] Derive $\partial J / \partial V_{ij}$.